

Manual do Sistema de Avaliação de Desempenho

SENAR Ceará

Sistema web (Flask + banco de dados Postgres/Supabase) para conduzir o ciclo anual de avaliação de desempenho: autoavaliação do empregado + avaliação do gestor imediato, cálculo do resultado final, liberação para o empregado, recursos (contestação de nota) e exportação de relatórios em Excel e PDF.

Este manual cobre: visão geral, instalação do zero, configuração, banco de dados, rotina de uso (RH/administração), estrutura de pastas e solução de problemas comuns.

Sumário

1. Visão geral do fluxo
2. Requisitos
3. Instalação do zero
4. Configuração do banco de dados (Supabase)
5. Variáveis de ambiente (.env)
6. Rodando o sistema localmente
7. Cadastro inicial (setores, funcionários, vínculos, ciclo)
8. Uso do dia a dia — Administração
9. Uso do dia a dia — Gestor e Empregado
10. Recursos (contestação de nota)
11. Exportações (Excel e PDF)
12. Estrutura de pastas e arquivos
13. Deploy em produção (Vercel)
14. Backup e versionamento (Git)
15. Solução de problemas comuns
16. Segurança — cuidados importantes

1. Visão geral do fluxo

1. O RH cria o ciclo de avaliação do exercício (ex.: 2026).
2. O RH cadastra os funcionários (nome, cargo, setor, matrícula, data de admissão) e os vínculos (quem avalia quem naquele ciclo) — manualmente ou importando planilha.
3. Cada gestor entra no sistema e vê a lista dos empregados que avalia.
4. Cada empregado entra e preenche a autoavaliação.
5. O formulário muda de acordo com o nível hierárquico da pessoa avaliada (Empregado, Gestor ou Superintendente têm formulários com fatores diferentes).
6. Depois que autoavaliação e avaliação do gestor estão concluídas, o sistema calcula o Resultado Final:

$$\text{Resultado} = \text{Autoavaliação} \times 0,30 + \text{Avaliação do Gestor} \times 0,70$$

Conceito: A (8,0 a 10,0) · B (7,0 a 7,9) · C (abaixo de 7,0)

7. O resultado fica visível só para a Administração até ela liberar, individualmente, para cada empregado ver o próprio resultado.
8. Se o empregado discordar do resultado, pode abrir um recurso, que passa pela Comissão → Gestor → Comissão → (se mantido) Presidência.

2. Requisitos

- Python 3.11+ instalado.
- Uma conta no Supabase (banco Postgres gratuito) — ou outro Postgres acessível via connection string.
- Git (para clonar/atualizar o código).
- (Opcional) Conta de e-mail Gmail, se quiser que a Comissão receba aviso por e-mail quando um recurso for aberto.

3. Instalação do zero

```
# 1. Clonar o repositório
git clone <url-do-repositorio>
cd avaliacao_desempenho

# 2. Criar e ativar o ambiente virtual
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# 3. Instalar as dependências
pip install -r requirements.txt
```

Dependências principais (arquivo requirements.txt):

Pacote	Para que serve
Flask	Framework web
Flask-SQLAlchemy / SQLAlchemy	Acesso ao banco de dados
psycopg2-binary	Driver de conexão com Postgres
python-dotenv	Carrega o arquivo .env
pandas	Leitura de planilhas na importação em lote
openpyxl	Geração dos arquivos Excel (.xlsx)
reportlab	Geração dos arquivos PDF

4. Configuração do banco de dados (Supabase)

1. Crie um projeto em supabase.com.
2. Vá em SQL Editor e rode os scripts da pasta sql/, nesta ordem (cada um depende do anterior):

Ordem	Arquivo	O que faz
1	001_schema.sql	Cria todas as tabelas do banco
2	002_seed_fatores.sql	Cadastra os 3 formulários (Empregado, Gestor, Superintendente) e seus fatores oficiais
3	003_setores.sql	Cria a tabela de setores/departamentos
4	004_nivel_hierarquico.sql	Adiciona o nível hierárquico ao funcionário
5	005_dados_pessoais.sql	Adiciona CPF, nascimento, sexo e salário
6	006_data_admissao.sql	Adiciona a data de admissão
7	007_elegibilidade_avaliacao.sql	Permite marcar manualmente se alguém participa ou não do ciclo
8	008_nivel_hierarquico_define_formulario.sql	Separa "cargo real" de "nível hierárquico"
9	009_login_funcionario.sql	Login individual (e-mail + senha)
10	010_prazos_avaliacao.sql	Prazos separados para autoavaliação e avaliação do gestor
11	011_resultados_finais.sql	Controle de liberação do resultado final
12	012_recursos.sql	Fluxo de recursos (contestação de nota)
13	013_recurso_comissao_inicial.sql	Ajuste do fluxo: Comissão entra logo na abertura do recurso

Se for um banco novo, rode todos em sequência. Se for atualizar um banco já existente com uma nova versão do sistema, rode só os scripts que ainda não foram aplicados (normalmente os mais recentes).

3. Vá em Project Settings → Database → Connection string → URI e copie a connection string (prefira o modo Session pooling).

5. Variáveis de ambiente (.env)

Copie o arquivo de exemplo e preencha com os seus dados:

```
cp .env.exemplo .env
```

Variável	Descrição
DATABASE_URL	Connection string do Supabase (passo anterior)
SECRET_KEY	Chave aleatória usada pelo Flask para sessão. Gere uma string qualquer, única e secreta
ADMIN_PASSWORD	Senha de acesso à área de Administração — só o RH deve ter
COMISSAO_PASSWORD	Senha de acesso à área da Comissão de Recursos
MAIL_USERNAME	(Opcional) E-mail do Gmail usado para avisar a Comissão de recurso novo
MAIL_APP_PASSWORD	(Opcional) "Senha de app" gerada em myaccount.google.com/apppasswords — não é a senha normal da conta
FLASK_DEBUG	1 em desenvolvimento (mostra erros detalhados); deixe 0 ou remova em produção

Nunca commita o arquivo .env real no Git — ele deve conter senhas e credenciais. Só o .env.exemplo (com valores fictícios) vai para o repositório.

6. Rodando o sistema localmente

```
python run.py
```

Acesse <http://localhost:5000> no navegador.

O terminal mostra os logs de cada requisição. Se algo der errado, o erro completo (traceback) aparece ali — é a primeira coisa a olhar quando um recurso "dá erro".

Para parar o servidor: Ctrl + C.

7. Cadastro inicial (setores, funcionários, vínculos, ciclo)

Acesse a área de Administração (/admin, com a ADMIN_PASSWORD).

1. Setores (/admin/setores) — cadastre os departamentos da empresa.
2. Funcionários (/admin/funcionarios) — cadastre nome, cargo, setor, e-mail, matrícula e data de admissão. Duas formas: um a um pelo formulário da própria tela, ou importando planilha (.xlsx/.csv) usando o modelo em exemplos/funcionarios_exemplo.csv (colunas: nome, cargo, setor, email, matricula). O setor é criado automaticamente se ainda não existir.
3. Vínculos (/admin/vinculos) — defina quem avalia quem. Também aceita importação em lote via exemplos/vinculos_exemplo.csv (colunas: avaliado, avaliador, com os nomes exatamente como cadastrados nos funcionários).
4. Ciclo de avaliação — crie o ciclo do exercício (ex.: 2026) e deixe-o com status "aberto" enquanto as pessoas preenchem as avaliações.

8. Uso do dia a dia — Administração

Principais telas dentro de /admin:

- Painel — visão geral do andamento do ciclo.
- Andamento — status de cada avaliação (quem já respondeu, quem falta).
- Resultados (/admin/resultados) — tela de "Resultado final da avaliação": lista todo mundo com matrícula, admissão, nota da autoavaliação, nota do gestor, resultado final ponderado e conceito (A/B/C). Tem busca por nome, filtro para mostrar só quem já concluiu, e botões para baixar o PDF e a planilha Excel consolidados (seção 11) — além de baixar o PDF/Excel individual de cada pessoa e liberar/bloquear a visualização do resultado para o próprio empregado.
- Recursos — telas separadas para Comissão, Gestor e Presidência acompanharem contestações de nota (seção 10).

9. Uso do dia a dia — Gestor e Empregado

- O gestor entra, escolhe seu nome na lista e vê os empregados que avalia, com o status de cada avaliação (pendente/rascunho/concluída).
- O empregado entra, escolhe seu nome e preenche a autoavaliação, no formulário do seu próprio nível hierárquico.
- As respostas podem ser salvas como rascunho (dá pra continuar depois) ou concluídas (fecha a avaliação para edição).
- Depois que a Administração libera, o empregado vê seu Resultado Final em "Minha área".

10. Recursos (contestação de nota)

Se o empregado discordar do resultado, pode abrir um recurso. O fluxo atual (documentado também em LEIA-ME.txt) é:

1. Empregado abre o recurso → vai para a Comissão.
2. Comissão manda mensagem ao gestor pedindo reavaliação.
3. Gestor responde (mantém a nota ou revisa) → volta para a Comissão (não vai direto para o empregado). Se o gestor revisou: a Comissão só confirma e encerra. Se o gestor manteve: a Comissão repassa a resposta ao empregado.
4. Empregado vê a resposta (só depois que a Comissão repassou) e decide: aceitar ou recorrer.
5. Se aceitar: vai para a Comissão confirmar o encerramento (não encerra sozinho).
6. Se recorrer: vai para a Comissão encaminhar à Presidência.
7. Presidência decide (acata ou não) — é a última instância, encerra direto.

A tela da Comissão tem 4 filas: recursos recém-abertos, gestor respondeu, empregado aceitou, e gestor manteve + empregado recorreu.

11. Exportações (Excel e PDF)

Na tela Resultado final (/admin/resultados):

- Por pessoa — cada linha tem os ícones de baixar o PDF e o Excel individuais (formulário completo com fatores, soma, média e assinatura).
- Consolidado (todo mundo do ciclo) — os botões "Baixar PDF" e "Baixar planilha (.xlsx)" no topo da tabela geram um único arquivo com todos os funcionários, no formato:

MAT	NOME	ADMISSÃO	NOTA	CONCEITO
-----	------	----------	------	----------

seguido da tabela de Referência × Real dos conceitos (percentual esperado de A/B/C vs. o que de fato aconteceu no ciclo).

Por padrão, essas exportações trazem só quem já concluiu a avaliação (mesma regra do filtro da tela). Para trazer todo mundo, inclusive quem está pendente, adicione `&somente_concluidos=0` na URL.

As metas de referência (hoje 20% / 65% / 15% para A/B/C) ficam nas constantes `META_CONCEITO_A`, `META_CONCEITO_B` e `META_CONCEITO_C`, no topo de `app/excel_resultado_final.py` e `app/pdf_resultado_final.py` — ajuste os dois arquivos juntos se a meta mudar em outro exercício.

A coluna `MAT` usa a matrícula cadastrada no funcionário; se estiver vazia, usa um número sequencial (1, 2, 3...) como a planilha antiga fazia.

12. Estrutura de pastas e arquivos

```
avaliacao_desempenho/
├── app/
│   ├── __init__.py          # cria a aplicação Flask, registra as rotas
│   ├── config.py            # lê as variáveis do .env
│   ├── extensions.py        # instância do SQLAlchemy
│   ├── models.py            # tabelas do banco (Funcionario, Avaliacao,
etc.)
│   ├── utils.py              # funções auxiliares (cálculo do resultado,
formatação)
│   ├── resultado_final_service.py # monta os dados do resultado final
(individual)
│   ├── excel_resultado_final.py # gera os Excel (individual e consolidado)
│   ├── pdf_resultado_final.py   # gera os PDF (individual e consolidado)
│   ├── pdf_avaliacao.py         # gera o PDF do formulário de avaliação
│   ├── pdf_timbrado.py          # cabeçalho/timbre usado em todos os PDFs
│   ├── email_service.py        # envio de e-mail (aviso à Comissão)
│   └── routes/
│       └── main.py             # tela inicial, login, seleção de
gestor/empregado
│   ├── avaliacoes.py          # preencher/salvar o formulário de avaliação
│   ├── admin.py                # cadastros, ciclo, resultados, exportações
│   ├── recursos.py             # fluxo de contestação de nota
│   ├── templates/              # HTML (Jinja2)
│   ├── static/
│   │   ├── css/                # estilo
│   │   └── img/                # logos
├── sql/                        # scripts de banco, em ordem (seção 4)
├── exemplos/                   # CSVs modelo para importação em lote
├── requirements.txt             # dependências Python
├── run.py                       # roda o servidor localmente
├── wsgi.py                     # ponto de entrada usado em produção (Vercel)
├── vercel.json                  # configuração de deploy na Vercel
├── .env.exemplo                 # modelo de variáveis de ambiente
├── README.md                    # visão geral resumida do projeto
└── LEIA-ME.txt                  # changelog do fluxo de recursos
```

13. Deploy em produção (Vercel)

O projeto já tem `wsgi.py` e `vercel.json` prontos para deploy na Vercel:

1. Suba o repositório para o GitHub (se ainda não estiver lá).
2. Importe o repositório em vercel.com.
3. Configure as mesmas variáveis de ambiente da seção 5 (DATABASE_URL, SECRET_KEY, ADMIN_PASSWORD, COMISSAO_PASSWORD, e as de e-mail se for usar) no painel da Vercel, em Project Settings → Environment Variables.
4. A Vercel detecta o wsgi.py e usa vercel.json para configurar a função serverless.
5. Após o deploy, a URL do projeto fica disponível (ex.: sistema-avaliacao-desempenho-psi.vercel.app).

Em produção, não deixe FLASK_DEBUG=1 — isso expõe detalhes internos do sistema em caso de erro.

14. Backup e versionamento (Git)

Fluxo básico depois de alterar arquivos do sistema:

```
git status          # ver o que mudou
git add <arquivos>  # ou "git add -A" para tudo
git commit -m "Descrição da mudança"
git push
```

Boas práticas:

- Nunca commitar o .env real, senha.txt ou qualquer arquivo com credenciais — só o .env.exemplo com valores fictícios.
- Fazer commits pequenos e descritivos (mais fácil de entender o histórico depois).
- Antes de subir uma mudança grande (ex.: alteração no banco), avise quem mais usa o sistema, e rode o script SQL correspondente antes de colocar o código novo no ar.

15. Solução de problemas comuns

Sintoma	Causa provável	O que fazer
Erro de conexão com o banco ao iniciar	DATABASE_URL errada ou vazia no .env	Conferir a connection string no painel do Supabase
Página fica em branco / erro 500	Ver o traceback no terminal onde o python run.py está rodando	Copiar a última parte do erro para investigar a causa
Login não funciona	Senha errada, ou variável não configurada no .env	Conferir o .env e reiniciar o servidor
Coluna "Mat." aparece vazia	Funcionário sem matrícula cadastrada	Preencher a matrícula no cadastro (ou importar em lote)
Resultado final não aparece	Falta autoavaliação e/ou avaliação do gestor concluída	Conferir na tela "Andamento" quem ainda não respondeu
Mudança no código não aparece	Servidor não recarregou	Reiniciar com Ctrl+C e python run.py de novo
Erro ao importar planilha	Colunas com nome diferente, ou nome/cargo com grafia diferente do já cadastrado	Comparar com os modelos em exemplos/

16. Segurança — cuidados importantes

- ADMIN_PASSWORD e COMISSAO_PASSWORD dão acesso a dados sensíveis (avaliações, salários, CPF). Compartilhe só com quem precisa.
- Nunca deixe o arquivo .env (ou qualquer arquivo com senha real, como senha.txt) dentro do repositório Git público.
- Se enviar o projeto para alguém analisar (ex.: em um .zip), remova antes os arquivos .env, senha.txt e qualquer coisa com credencial real — ou troque as senhas depois, por segurança.
- O MAIL_APP_PASSWORD é uma senha de uso exclusivo do sistema para enviar e-mail — pode ser revogada a qualquer momento em myaccount.google.com/apppasswords, sem afetar a senha normal da conta.

